

Data Science Study Guide

Corrected version

How to read this: **Red bold** = something in your original notes was wrong and has been fixed. **Green** = short addition that fills a gap likely to show up on a test. Everything else is your original wording (lightly cleaned up for typos). Your notes had Anonymity and "Git Cmds" twice, so each is merged into one section.

(P)ersonal (I)dentifiable (I)nformation

- **Minimization:** Only collect/keep the data needed for the purpose (e.g., removing columns that don't assist analysis)
 - May lose context or show bias, but balances utility + privacy
- **Anonymization techniques**
 - **Suppression:** Eliminating identifiable data
 - **Masking:** Hiding/blurring part of a value while keeping its structure (e.g., ***-**-1234)
 - **Pseudonymization:** **Replacing identifiers with a pseudonym/token. It is reversible with a separate key**, so it is NOT full anonymization
 - **Generalization:** Reducing variability by grouping and categorizing (age 34 → 30–39)
 - **Synthetic Data:** Creating new fake values with the same statistical properties

Anonymity

Can info about an individual be singled out? If yes, we must make it blend in.

- **Quasi-identifier:** **Info that does not identify you alone, but can when combined** (ZIP code + age + gender)
- **Sensitive attribute:** Information that should not be revealed (diagnosis, salary)
- **K-Anonymity:** **Every record matches at least k-1 other records** on its quasi-identifiers, so each person blends into a group of at least k
 - Groups are created based on quasi-identifiers
- **L-Diversity:** **Each k-anonymous group must contain at least l different sensitive values**
 - Fixes k-anonymity's weakness: if everyone in a group has the same diagnosis, the attacker learns it anyway (homogeneity attack)
- **T-Closeness:** **The distribution of the sensitive attribute** in each group should not be too different from its distribution in the whole data set (within a distance t)
- **Mondrian Algorithm:** Recursively splits data at the median of a quasi-identifier into smaller groups, stopping when a split would make a group smaller than k

HTTP Methods & Status Codes (your notes called this "Git Cmds")

These are HTTP request methods used by APIs, not Git commands.

- **GET:** Retrieve a resource
- **POST:** Create a new resource
- **PUT:** Update or replace a resource (whole thing)
- **PATCH:** Partially update a resource
- **DELETE:** Remove a resource
- **Status codes**

- 200: OK
- 201: Created (successful POST)
- 400: Bad Request (**your second copy said 404 for this**)
- 401: Unauthorized (not authenticated) | 403: Forbidden (authenticated but not allowed)
- 404: **Not Found** (the resource doesn't exist)

Authentication v. Authorization

'Who is this user?' v. 'What is this user allowed to do?'

- **API Authentication:** Protects API from unauthorized access
 - Ensure secure data exchange Server $\longleftrightarrow$ Client
 - Encrypted w/ HTTPS (Hypertext Transfer Protocol Secure)
- **Bearer Token:** A 'proof pass' sent to the server **in place of** the user's credentials, so they don't have to be sent with every request (**whoever holds/bears it gets access**)
- **OAuth 2.0:** Uses 'Access Tokens' with a defined scope; allows 3rd-party access without sharing credentials
 - 3rd Party: performs a task on your behalf without knowing your sensitive information
 - **OAuth 2.0 is an authorization framework (what you can access), not authentication**

OAuth 2.0 Actors / Flow

- 1) Client $\rightarrow$ (Authorization Request) $\rightarrow$ Resource Owner $\rightarrow$ (**Authorization** Grant) $\rightarrow$ Client
- 2) Client $\rightarrow$ (Auth Grant) $\rightarrow$ Authorization Server $\rightarrow$ (Access Token) $\rightarrow$ Client
- 3) Client $\rightarrow$ (Access Token) $\rightarrow$ Resource Server $\rightarrow$ (Resources) $\rightarrow$ Client

Principle of Least Privilege

Give the client the minimum amount of access/resources necessary to perform a task.

- Controls what is returned by a given endpoint
- Changes with the user's scope

(A)pplication (P)rogramming (I)nterface

A set of rules that lets software programs communicate (e.g., how an app talks with a server)

- Client (Request) $\rightarrow$ API (forwards request) $\rightarrow$ Server $\rightarrow$ API (Response) $\rightarrow$ Client

Apache Subversion (SVN)

Saves versions of a project. 'Commit' saves the current state.

- **Centralized** version control: one central server holds the history; commits go straight to it

Git – (V)ersion (C)ontrol (S)ystem

*Tracks changes to your files over time. **Distributed**: every clone has the full history.*

- **ADD: Stages changes** (puts them in the staging area) so they're included in the next commit
- **COMMIT:** Saves a snapshot of the staged changes to the local repository
- **PUSH:** Uploads your local commits to a remote repository
- **PULL:** Retrieves updates from the remote repo and merges them into your local branch (**pull = fetch + merge**)

- **CLONE:** **Copies a remote repository onto your machine** as a new local repository
- **STATUS:** Checks status of files (modified, staged, untracked)
- **FETCH:** Downloads new **commits** from the remote without merging them into your work (safe)
- **Branch:** Separate line of development from the main version
- **Pull Request:** A request to merge **changes from one branch into another (e.g., into main)**, allowing code review before the merge

Code Review

Correctness, design/style, understanding, testing, and other parts of the code are reviewed.

- **Docstring:** Text within code (first line of a function/class/module) that explains its functionality

Software Licenses

- **Copyleft:** **Free to use, modify, and share, BUT modified versions you distribute must use the same copyleft license** (e.g., GPL)
- **Permissive:** Free use; modifications do NOT have to be open-source
 - Both permissive and copyleft licenses disclaim warranty/liability
- **Proprietary:** **Vendor keeps the rights**; use/modification is not permitted unless authorized by the vendor
- **Examples**
 - MIT License (permissive)
 - Berkeley Software Distribution / BSD (permissive): **You can't use the original authors' names to endorse or promote your derived product**
 - Apache 2.0 (permissive): Explicit patent grant (protection) + must include license/NOTICE and state changes
 - MATLAB (proprietary)

Project Organization / Documentation

- **README:** **Plain** text (often Markdown) file describing the project: what, why, how to use it
- **Installation:** Requirements to properly run it, including setup (venv code)
- **Quick Start:** How to quickly get it running
- **src/:** Folder containing code, data-loading files, and other source code
- **config/:** Settings the app needs to run
- **Environment/Requirements file** (e.g., requirements.txt)
- **Documentation of classes/functions** (docstrings)

Virtual Computing

- **Local System:** Convenient and cheap; useful for low-frequency, low-compute work
- **Serverless:** Running code in the cloud without managing servers. The cloud provider handles management, infrastructure, and scaling, so developers focus on application code

Types of Computing (AWS)

- **Lambda:** **Serverless functions:** run short, event-triggered code (max 15 min); pay per run
- **Fargate:** **Serverless compute for containers:** run containers without managing servers
- **EC2 (Elastic Compute Cloud):** Virtual servers (instances) you choose and manage

- Tailor infrastructure to the workload through instance types → scalability
- Most control/flexibility, but you manage it
- **SageMaker**: Managed machine-learning service (**Jupyter** notebooks)
 - Train, deploy models, and make visualizations
 - Runs on EC2 instances behind the scenes, but does NOT have the flexibility of EC2

Shared Responsibility Model

- Customer responsible for security 'IN' the cloud (your data, access settings)
- AWS responsible for security 'OF' the cloud (hardware, data centers)
- **(I)ntity and (A)ccess (M)anagement (IAM)**
 - User: Individual requiring long-term access
 - Group: Collection of users who need the same access
 - Role: Temporary credentials for access

Simple Storage Service (S3)

- S3 + SQL: class foundation for storing data to train models
- **Buckets**: Top-level cloud storage container holding objects
 - **Standard**: Frequent access, fast reads/writes
 - **Glacier**: Cheaper archive storage; retrieval takes minutes up to ~12 hrs (**Deep Archive**: up to 48 hrs)
- **Undoing deletions**: **Only possible in S3 if Versioning is turned on** (deleted objects can be restored)
 - **ROLLBACK** is a SQL command that undoes changes in a transaction that has not been committed

AWS S3 Commands

The commands start with **aws s3** (lowercase)

- **mb**: Make (create) a bucket
- **cp**: Copy files **to/from S3 (upload or download)**; add --recursive for folders
- **mv**: Move or rename files/folders
- **ls**: List buckets or the contents of a bucket
- **rm**: Delete objects | **sync**: Copy only new/changed files between a folder and a bucket

(S)tructured (Q)uery (L)anguage

- **Relational (SQL)**: **Data organized in tables (rows and columns)**
 - **Designed to reduce redundancy** (normalization) rather than store it
 - Tables connect by linking foreign keys to primary keys
- **Non-Relational (NoSQL)**: **Flexible, non-table formats**: documents (JSON, hierarchical), key-value, graph (networks), wide-column

Steps to Deployment

- **Model Registration**
 - Version control for models: access to previous working models
 - AWS SageMaker Model Registry
- **Endpoints**

- **A URL where the deployed model receives requests and returns predictions**
- Data arrives as raw bytes; it's converted to readable data for the model to predict on
- APIs govern access to the URL
 - POST (HTTP) → Step 1: parse the HTTP request → Step 2: run model and return response
- **API Gateway**
 - Manages, secures, and coordinates API traffic for the client
 - A centralized entry point with proper security

Right to Explanation

You have the right to understand what AI/automated decisions have done with your data (tied to the EU GDPR)

Human in the Loop

Required human participation (operation, supervision, decision making)

- Humans must be **ACCURATE** to catch mistakes and add oversight
- Inner workings of the decision are clear and **TRANSPARENT**
- **ACCOUNTABILITY** falls on humans

EU AI Act

*The world's 1st **comprehensive** law regulating AI; **human oversight is required for high-risk systems***

- Risk pyramid with highest risk at the top:
 - Unacceptable (banned) → High (strict rules + human oversight) → Limited (transparency, e.g., say it's a chatbot) → Minimal (no rules)